

Homework 3

1、设计一个递归算法求解组合问题：从自然数 $1, 2, \dots, m$ 中任取 k 个数的所有组合。

解：设 $\text{comb}(m,k)$ 为从 $1 \sim m$ 中任取 k 个数的所有组合。当组合的第 1 个数字选定后，其后的数字是从余下的 $m-1$ 个数中取 $k-1$ 个数的组合，即 $\text{comb}(m-1,k-1)$ ， $\text{comb}(m,k)$ 为“大问题”， $\text{comb}(m-1,k-1)$ 为“小问题”。

采用 a 数组保存组合结果，由于每个组合不考虑次序，为了简便，不妨设定结果按照递增排列，如求得 $1, 2, 3$ 中的两个数的所有组合为： $12, 13, 23$ 。先确定 a 中最后一个元素的取值，对于 $a[k-1]$ 只能取 $m \sim k$ 中的一个值 i （这样 $a[0..k-2]$ 的取值均小于 i ），如图 3.24 所示，再组合 $\text{comb}(i-1,k-1)$ 的结果便得到了一个组合，如 $1, 2, 3$ 中的 2 个数的所有组合的过程如图 3.25 所示。其递归模型如下：

$\text{comb}(m,k) \equiv$ 输出一个解

当 $k=1$ 时

$\text{comb}(m,k) \equiv a[k-1]$ 位置取 $m \sim k$ 的任何值 i

其他情况

并组合 $\text{comb}(i-1,k-1)$ 的结果

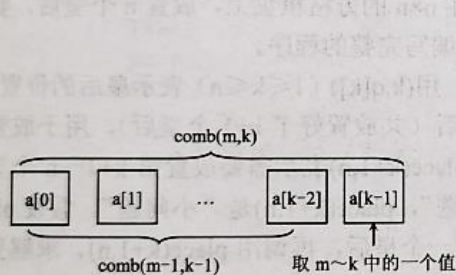


图 3.24 求组合问题

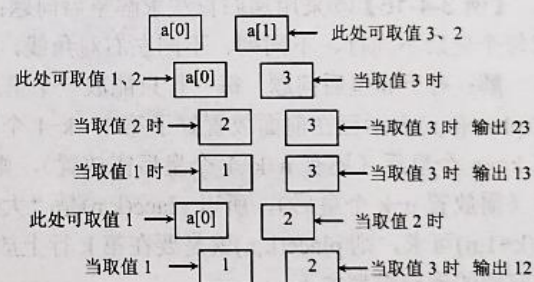


图 3.25 求 $\text{comb}(3,2)$ 的过程

对应的完整程序如下：

```
#include <stdio.h>
#define MaxSize 10
int m,k; //全局变量
void print(int a[]) //输出一个组合
{
    int j;
    printf(" ");
    for (j=0;j<=k-1;j++)
        printf("%d ",a[j]);
    printf("\n");
}
void comb(int a[],int m,int k)
{
    int i;
```

```

    for (i=m;i>=k;i--)
    {   a[k-1]=i;
        if (k>1)
            comb(a,i-1,k-1);
        else
            print(a);
    }
}
void main()
{   int a[MaxSize];
    m=5; k=3;
    printf("1..%d 中 %d 个的组合结果如下:\n",m,k);
    comb(a,m,k);
    printf("\n");
}

```

//确定 a[k-1] 位置的取值
//递归调用
//k=1 时输出一个组合

本程序的执行结果如下：

1..5 中 3 个的组合结果如下：

```

3 4 5
2 4 5
1 4 5
2 3 5
1 3 5
1 2 5
2 3 4
1 3 4
1 2 4
1 2 3

```

2、给出进入结构和离开结构的一些整数的次序（假定在栈和队列中都是整数），请确定这是怎样的数据结构，是栈还是队列？

注意：设定在所有的整数没有进入数据结构前，没有整数被弹出。

输入包含多个测试用例，输入的第一行是一个整数 $T(T \leq 100)$ ，表示测试用例的个数，然后给出 T 个测试用例。

每个测试用例包含 3 行：测试用例的第一行给出一个整数 N ，表示整数个数($1 \leq N \leq 100$)。测试案例的第二行给出用空格分隔开的 N 个

整数，表示进入结构的次序（即第一个数据最先进入）。测试案例第三行给出用空格分隔开的 N 个整数，表示离开结构的次序（第一个数据是最先离开）。

对于每个测试用例，在一行中输出判定的结果。如果结构只能是一个栈，则输出“**stack**”；如果结构只能是一个队列，则输出“**queue**”；如果结构既可以是栈，也可以是队列，则输出“**both**”，否则输出“**neither**”。

示例：

样例输入	样例输出
4	stack
3	queue
1 2 3	both
3 2 1	neither
3	
1 2 3	
1 2 3	
3	
1 2 1	
1 2 1	
3	
1 2 3	
2 3 1	

按照出入结构的次序，若第 i 个进入结构的元素在第 i 个离开，则该结构满足“先进先出”的性质，则该结构为队列；若第 i 个进入结构的元素在倒数第 i 个离开，则该结构满足“先进后出”的性质，则该结构为栈 ($0 \leq i \leq n-1$)。

设： $a[i]$ 为第 i 个进入结构的整数； $b[i]$ 为第 i 个离开结构的整数 ($0 \leq i \leq n-1$)； $issta$ 为结构的栈标志； $isque$ 为结构的队列标志。初始时， $issta$ 和 $isque$ 设为 true。

判别结构性质的方法如下。

依次搜索 a 和 b 的每个元素，判断结构中的所有元素是否具备队列或栈的性质：若 $b[i] \neq a[i]$ ，则结构不符合“先进先出”的特性，该结构不是队列 ($isque=false$)；若 $b[i] \neq a[n-i-1]$ ，则结构不符合“后进先出”的特性，该结构不是栈 ($issta=false$)，其中 $0 \leq i \leq n-1$ 。

最后根据 $issta$ 和 $isque$ 的值确定结构的性质，如右表：

issta	isque	输出
false	false	"neither"
false	true	"queue"
true	false	"stack"
true	true	"both"



参考程序

```
#include <iostream>
using namespace std;
const int maxn = 100 + 10;
int main(void)
{
    int loop;
    cin >> loop;
    while (loop--) {
        int n, a[maxn];
        cin >> n;
        for (int i = 0; i < n; i++)
            cin >> a[i];
        bool isque = true, issta = true; // 队列和栈标志初始化

        for (int i = 0; i < n; i++) {
            int x;
            cin >> x;
            if (x != a[i])
                isque = false;
            if (x != a[n - i - 1])
                issta = false;
        }

        if (issta && isque)
            cout << "both" << endl;
        else if (issta)
            cout << "stack" << endl;
        else if (isque)
            cout << "queue" << endl;
        else
            cout << "neither" << endl;
    }
    return 0;
}
```